Lab 13 — Hands-on lab: the web → api chain in the cluster

Goal: deploy the front end and the API, watch nginx die for lack of a Service, create the `api` Service and observe DNS, cluster IP and EndpointSlices, expose the front end as a NodePort then a LoadBalancer, and finally go through the Traefik Ingress on port 80 — and check that the endpoints follow the Pods.

Prerequisites — Lab 12 completed: k3s, `localhost/api:1.0` in the cluster, Podman with `api:1.0`.

Files provided (`files/`) - `Dockerfile.web`, `nginx-proxy.conf`, `index.html` — the `web:1.0` image: nginx + the front end + the `/api/` proxy → `http://api:8080/`. - `deployment-api.yaml`, `deployment-web.yaml` — the two Deployments (the API's is the one from lab 12). - `service-api.yaml` (ClusterIP), `service-web.yaml` (NodePort 30080), `ingress-web.yaml` (Traefik, path `/`).

Step 0 — The front-end image in the cluster

```
mkdir -p ~/labo-docker/13 && cd ~/labo-docker/13 && cp <path-to-the-lab>/files/* .
podman build -q -f Dockerfile.web -t web:1.0 .
podman save localhost/web:1.0 | sudo k3s ctr images import -
sudo k3s crictl images | grep -E 'localhost/(api|web)'
```

Observe `localhost/web 1.0` next to the `localhost/api` images in the cluster.

Step 1 — No Service, no name

```
kubectl create namespace lab13 ; kubectl config set-context --current --namespace=lab13
kubectl apply -f deployment-api.yaml -f deployment-web.yaml
kubectl rollout status deployment/api --timeout=60s
sleep 20 ; kubectl get pods
kubectl logs deploy/web --tail=1
```

Observe the API `Running` and `web` in `CrashLoopBackOff` with `RESTARTS 2`, and the cause: `nginx: [emerg] host not found in upstream "api" in /etc/nginx/conf.d/default.conf:15`.

Explanation. In lab 07, the Podman network named every container. Here, a Pod has no DNS name: nginx resolves `api` at startup, finds nothing, and exits. Only a Service carries a name.

Step 2 — The ClusterIP Service

```
kubectl apply -f service-api.yaml
kubectl rollout status deployment/web --timeout=120s
kubectl get pods
kubectl get svc api ; kubectl get endpointslices
```

Observe `web` turned `1/1 Running` **without any command from you** (`RESTARTS 3`, the backoff eventually succeeded); a Service `api ClusterIP 10.43.x.x <none> 8080/TCP`; and an EndpointSlice `api-xxxxx` with the two IPs of the API Pods.

```
kubectl exec deploy/web -- wget -qO- http://api:8080/actuator/health ; echo
kubectl exec deploy/web -- cat /etc/resolv.conf
kubectl exec deploy/web -- wget -qO- http://api.lab13.svc.cluster.local:8080/ ; echo
kubectl scale deployment api --replicas=3 ; sleep 8 ; kubectl get endpointslices
```

Observe `{"status":"UP"}` from the `web` Pod; a `resolv.conf` with `search lab13.svc.cluster.local svc.cluster.local cluster.local` and `nameserver 10.43.0.10`; the same answer through the full name; and, after scaling to three replicas, **three** addresses in the EndpointSlice.

Explanation. The Service has a virtual IP and a name; CoreDNS resolves the name, kube-proxy translates the IP to one of the endpoints, and the endpoint list follows the ready Pods.

Step 3 — NodePort: a door on the node

```
kubectl apply -f service-web.yaml ; sleep 3 ; kubectl get svc web
NODE=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[0].address}') ; echo $NODE
curl -s http://$NODE:30080/ | head -3
curl -s http://$NODE:30080/api/actuator/health ; echo
```

Observe `web NodePort 10.43.x.x <none> 80:30080/TCP`, your node's IP (the WSL distribution's), the front end's `<!doctype html>` and `{"status":"UP"}` through the nginx proxy — all on the node's port `30080`. From Windows, `http://localhost:30080/` opens the same page.

Explanation. A NodePort opens the same port on every node and relays it to the Service. Handy for testing; in production, an address without an exotic port is preferred.

Step 4 — LoadBalancer: k3s's ServiceLB

```
kubectl expose deployment web --name=web-lb --type=LoadBalancer --port=8081 --target-port=80
sleep 10 ; kubectl get svc web-lb
kubectl get pods -n kube-system -l svccontroller.k3s.cattle.io/svcname=web-lb -o name
curl -s http://$NODE:8081/api/actuator/health ; echo
kubectl delete svc web-lb
```

Observe `web-lb LoadBalancer 10.43.x.x <node-IP> 8081:3xxxx/TCP` — `EXTERNAL-IP` is the node's address —, a `svclb-web-lb-…` Pod in `kube-system`, and `{"status":"UP"}` on port `8081`.

Explanation. Bare Kubernetes would leave `<pending>`; ServiceLB simulates a load balancer by opening the port on the node. Port 80 was already taken by Traefik, hence `8081`.

Step 5 — The Ingress

```
kubectl get svc -n kube-system traefik
kubectl apply -f ingress-web.yaml ; sleep 5 ; kubectl get ingress
curl -s http://localhost/ | head -3
curl -s http://localhost/api/actuator/health ; echo
curl -s http://$NODE/api/ ; echo
```

Observe Traefik as a `LoadBalancer` on `80:3xxxx/TCP,443:3xxxx/TCP` with the node's IP; the `web` Ingress of class `traefik`, host `*`, `ADDRESS` = node IP; then the front end and the API on **port 80**, with no NodePort: `http://localhost/`.

Explanation. The Ingress describes "`/` → Service `web`"; Traefik, already on the node's port 80, applies it. Chain: port 80 → Traefik → Service `web` → nginx → (`/api/`) Service `api` → Pod. From Windows, `http://localhost/` works — k3s runs as root, the restriction on ports < 1024 does not apply.

Step 6 — Endpoints follow the Pods

```
POD=$(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}')
kubectl get endpointslices -l kubernetes.io/service-name=api -o
jsonpath='{.items[0].endpoints[*].addresses[0]}' ; echo
kubectl delete pod $POD ; sleep 1
kubectl get endpointslices -l kubernetes.io/service-name=api -o
jsonpath='{.items[0].endpoints[*].addresses[0]}' ; echo
kubectl port-forward svc/api 18080:8080 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080/actuator/health ; echo ; kill %1
```

Observe three addresses, then, one second after deleting a Pod, that Pod's address replaced by its successor's (the lab 12 ReplicaSet recreated it); and `port-forward` working towards a **Service** too.

Explanation. The Service does not know your Pods by name but by their labels and their `Ready` state: the list is recomputed on every change, with no intervention.

Clean-up

```
kubectl delete namespace lab13
kubectl config set-context --current --namespace=default
kubectl get svc -A | grep -c lab13
```

Observe `0`: Services, Ingress and Deployments went away with the namespace. Keep `web:1.0` and `api:1.0` in Podman and in the cluster: labs 14 and 15 complete the stack with configuration and the database.

What you must be able to state now

- Without a Service, nginx dies: `host not found in upstream "api"`; with the Service, it recovers on its own at the next backoff.
- A Service = `ClusterIP 10.43.x.x`, a name resolved by `10.43.0.10` through `search lab13.svc.cluster.local`, an EndpointSlice with the ready Pods' IPs.
- `scale` to 3 = three endpoints; a deleted Pod leaves the list and its replacement enters it.
- NodePort `30080` on the node's IP; LoadBalancer = ServiceLB, `EXTERNAL-IP` = node IP, one `svclb-…` Pod per Service.
- Ingress `web` → Traefik on the node's port 80 → `http://localhost/` and `http://localhost/api/actuator/health`.
- `port-forward svc/api` crosses everything without exposing anything.